**Проектирование БД** - процесс создания схемы данных и определения необходимых ограничений целостности.

Проектируемая база данных должна обеспечивать:

- хранение всей необходимой информации;
- возможность получения данных по всем необходимым запросам;
- сокращение избыточности и дублирования данных;
- целостность данных (правильность их содержания): исключение противоречий в содержании данных, исключение их потери и т.д.

Основными задачами при проектировании баз данных являются:

1. Определение границ проектируемой предметной области (постановка задания), построение структуры данных предметной области – концептуальное проектирование (инфологическая модель).
2. Выбор оптимальной СУБД для заданной предметной области. Описание БД на языке СУБД – логическое проектирование (даталогическая модель).
3. Определение типов используемых данных и методов доступа к ним (запросы, отчёты, формы) – физическое проектирование (физическая модель).

**Проектирование БД происходит в несколько этапов.**

**На первом этапе** проектирования баз данных производится системный анализ предметной области. Эта задача предшествует проектированию базы данных, и решают ее аналитики. Однако проектировщик баз данных должен знать результаты выполнения этой задачи и уметь правильно интерпретировать их в ходе проектирования. Результаты анализа предметной области, а именно, модели данных на семантическом уровне, являются исходными данными для решения задач проектирования базы данных. На данном этапе рассматриваются цели и задачи, анализируются информационные потребности всех категорий будущих пользователей базы данных. Рассматриваются формы входных и выходных потоков

данных. Затем уточняются алгоритмы и процедуры обработки хранимых данных. Формируются требования, которым должна удовлетворять проектируемая база данных, и определяется список объектов предметной области, свойства которых будут использоваться при разработке базы данных.

**На второй стадии** проектирования выполняется моделирование данных. **Моделирование данных** – это процесс создания логической структуры данных.

На третьем **этапе** производится выбор СУБД. Теоретически при выборе СУБД нужно принимать во внимание десятки факторов. Но практически разработчики руководствуются лишь собственной интуицией и несколькими наиболее важными критериями, к которым, в частности, относятся

- тип модели данных, которую поддерживает данная СУБД, её адекватность потребностям рассматриваемой предметной области;
- характеристики производительности системы;
- запас функциональных возможностей для дальнейшего развития ИС;
- степень оснащённости системы инструментарием для персонала администрирования данными;
- удобство и надежность СУБД в эксплуатации;
- стоимость СУБД и дополнительного программного обеспечения.

После того как выбор СУБД завершён, необходимо приступить к следующему, **четвертому этапу**, – проектированию даталогической модели базы данных. При формировании даталогической схемы каждая из определённых в концептуальной схеме сущностей отображается в таблицу, которая является одним отношением. При этом следует учитывать ограничения на размер таблиц, накладываемые конкретной СУБД.

**На пятом этапе** необходимо в конкретной СУБД, выбранной ранее, реализовать базу данных по той информации, которую собрали, обработали и подготовили (на предыдущих этапах проектирования базы данных). Описываются модули, их назначение, а также структура модулей.

**Цель инфологического проектирования** состоит в получении моделей, отражающих предметную область и информационные потребности пользователей. В качестве инструмента для построения модели данных на этапе проектирования можно выбрать модель «сущность–связь» (Entity–Relationship).

Метод «сущность–связь» по-другому называют методом «ER– диаграмм» (ER –аббревиатура от слов Entity (сущность) и Relation (связь)).

**Связи между сущностями**

В реляционных БД связи позволяют избежать избыточности данных. Связь работает путем сопоставления данных ключевых столбцов — обычно это столбцы с одинаковыми именами из двух различных таблиц. В большинстве случаев связь сопоставляет первичный ключ одной таблицы (уникальный идентификатор) с элементом внешнего ключа другой таблицы.

В зависимости от того, как определены поля связи основной и дополнительной таблиц (как соотносятся ключевые поля с полями связи), между двумя таблицами в общем случае могут устанавливаться следующие четыре основных вида связи:

- один – один (1:1);
- один – много (1:M);
- много – один (M:1);
- много – много (M:N).

Тип связи зависит от того, как определены связанные столбцы.

**Связь «один к одному»**

При связи «один к одному» одной строке таблицы A может соответствовать не более одной строки таблицы B (и наоборот). Связь «один к одному» создается, если оба связанных столбца являются первичными ключами или имеют ограничение, обеспечивающее их уникальность. Связи этого типа встречаются довольно редко, поскольку связанную подобным образом информацию обычно удается поместить в одной таблице.

**Связь «один ко многим»**

«Один ко многим» — наиболее распространенный тип связи. При этом типе связи одной строке таблицы A может соответствовать множество строк таблицы B, но любой строке таблицы B может соответствовать только одна строка таблицы A.

**Связь «многие ко многим»**

При связи «многие ко многим» одной строке таблицы A может соответствовать множество строк таблицы B (и наоборот). Такая связь создается путем определения третьей таблицы под названием «соединяющая таблица», первичный ключ которой состоит из внешних ключей таблиц A и B.

## Обеспечение целостности данных

Термином «целостность данных» обозначается такое состояние базы данных, когда все хранящиеся в ней данные верны. Если происходит сохранение в базе неверных данных, то говорят, что целостность данных потеряна.

Ссылочная целостность обеспечивает сохранность связей между таблицами при добавлении или удалении записей. В SQL Server ссылочная целостность основана на связях между внешними и первичными ключами или между внешними и уникальными ключами (через ограничения FOREIGN KEY и CHECK). Ссылочная целостность гарантирует согласованность значений ключа в связанных таблицах. Подобная согласованность требует отсутствия ссылок на несуществующие значения и согласованного изменения ссылок на ключ во всей базе данных при изменении самого ключа.

При обеспечении ссылочной целостности SQL Server предотвращает следующие действия пользователей:

- добавление записей в связанную таблицу, если нет необходимой записи в главной таблице;
- изменение значений в главной таблице, в результате которого в связанной таблице останутся «зависшие» записи;
- удаление записей из главной таблицы при наличии связанных записей во внешней таблице.

**Ограничения PRIMARY KEY**

В таблице обычно есть столбец (или комбинация столбцов), значения которых уникально идентифицируют каждую строку таблицы. Этот столбец (или столбцы) называются первичным ключом таблицы и обеспечивает целостность сущностей таблицы. Можно со щать первичный ключ, определив ограничение PRIMARY KEY при создании или изменении таблицы. В таблице может быть только одно ограничение PRIMARY KEY, причем столбец, который участвует в ограничении, не должен допускать пустых значений. Поскольку ограничения PRIMARY KEY гарантируют уникальность данных, они часто определяютси как столбцы с идентификатором. Когда в таблице определено ограничение PRIMARY KEY, SQL Server 2000 обеспечивает уникальность данных, создавая уникальный индекс для столбцов первичного ключа. Этот индекс также обеспечивает быстрый доступ к данным при использовании первичного ключа в запросах. Если ограничение PRIMARY KEY определено для нескольких столбцов, то в одном столбце значения могут повторяться, но каждая комбинация значений всех столбцов, которые входят в определение ограничения PRIMARY KEY, должна быть уникальной.

**Ограничения FOREIGN KEY**

Внешний ключ — это столбец или комбинация столбцов, которая используется для установления и обеспечения связи между данными двух таблиц. Связь между двумя таблицами устанавливается путем добавления к одной из таблиц столбца (столбцов) и определения для него ограничения FOREIGN KEY. Этот столбец содержит значения первичного ключа из второй таблицы. В таблице может быть несколько ограничений FOREIGN KEY

## Создание базы данных с рациональной структурой

1. *В таблице должен быть идентификатор*

Фундаментальное правило теории проектирования баз данных - в каждой таблице должен быть уникальный идентификатор строк, который представляет собой столбец или набор столбцов, позволяющий отличить записи таблицы друг от друга. Существование записей с одинаковыми значениями идентификатора недопустимо. Столбец (или набор столбцов), который служит уникальным идентификатором строк, является первичным ключом таблицы.

2. *В таблице должны храниться сведения лишь об одном типе объектов*

Если хранить в таблице слишком много информации, то эффективность и надежность хранения данных может снизиться.

*3. В таблицах следует избегать столбцов, допускающих пустые значения*

В таблицах могут быть столбцы, допускающие пустые значения. Пустое значение свидетельствует о том, что у этого атрибута записи значение отсутствует. Несмотря на то, что в единичных случаях пустые значения полезны, лучше всего использовать их как можно реже, поскольку их приходится обрабатывать специальным образом, что увеличивает сложность операций с данными. Если есть несколько столбцов, допускающих пустые значения и в нескольких строках эти столбцы содержат пустые значения, следует рассмотреть возможность выделения таких столбцов в отдельную таблицу, связанную с главной таблицей. Храня данные в двух отдельных таблицах, вы упростите структуру главной таблицы, не исключая возможности при необходимости сохранять и пустые значения.

*4. В таблице не должно быть повторяющихся значений или столбцов*

В таблице не должно быть списков значений для некоторой единицы информации. Допустим, требуется учитывать названия и авторов книг. Хотя большинство книг написаны одним автором, их может быть два и более. Если для хранения имени автора в таблице предназначен только один столбец, то возникает проблема. Можно, конечно, хранить имена обоих авторов в одном столбце, но при этом затруднится вывод списка авторов по отдельности. Другое решение - изменить структуру таблицы и добавить еще один столбец для хранения имени второго автора. Но это поможет, только если авторов не более двух. Если же их три, придется добавлять третий столбец.